

Programming Language Design

Module 12: More Monads

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

Last Time

We learned about the `IO` monad.

We learned how we could build a basic interpreter. The idea:

- Evaluating a lisp program means turning it into an `IO`
- We return the `IO` from `main`
- The runtime environment evaluates it.

This time

Monad prep! Our quiz is soon, so let's do more prep for it.

We will also learn some more monads. `IO` is not the only one!

In fact, several data types we're already pretty familiar with are also monads, such as lists, `Maybe`, and `Either`.

First, let's start with a classic problem that is often the first problem of a technical interview series...

FizzBuzz

Write a program that, for every integer from 1 to 100, outputs the following:

- "Fizz" if the number is divisible by 3
- "Buzz" if the number is divisible by 5
- "FizzBuzz" if both conditions are true
- The number itself if it is divisible by neither 3 nor 5.

FizzBuzz (2)

So the output is expected to be:

```
1
2
Fizz
4
Buzz
...
14
FizzBuzz
16
...
```

Classically, we put newlines after each thing we print.

Give it a try. Can you do it in Haskell? What about another language?

[Think about it]

Let's start with C

If this is a tough problem, [let's do it in C first]

C FizzBuzz

```
int main() {  
    for (int i = 1; i <= 100; i++) {  
        if (i % 15 == 0) puts("FizzBuzz");  
        else if (i % 3 == 0) puts("Fizz");  
        else if (i % 5 == 0) puts("Buzz");  
        else printf("%d\n", i);  
    }  
  
    return 0;  
}
```


C FizzBuzz remarks

We first check if `i % 15 == 0`, because that's the same as checking if `i` is divisible by 3 and 5 together.

This is kind of a trick, and it's okay if you didn't know it. You could do this just fine:

```
if ((i % 3 == 0) && (i % 5 == 0)) puts("FizzBuzz");
```

Sometimes, people prefer to print "Fizz" and "Buzz" independently:

```
if ((i % 3) == 0) printf("Fizz");  
if ((i % 5) == 0) printf("Buzz");  
if ((i % 3) != 0 && (i % 5) != 0) printf("%d\n", i);
```

This is fine, too, and is a nice design (it would scale well if the interviewer added more factors)

Why is it hard in Haskell?

We need to do these things:

- Loop through all the numbers (from 1 to 100)
- Determine whether a number is a "Fizz", a "Buzz", or just itself
- Print that

The first step is actually the hardest in Haskell. We don't have *loops*. Or do we?

It turns out, with monads, we do.

But first, let me give *one more* example, in one more language....

Python FizzBuzz

```
def fizzify(i):  
    if i % 15 == 0: return "FizzBuzz"  
    elif i % 3 == 0: return "Fizz"  
    elif i % 5 == 0: return "Buzz"  
    else: return i  
  
for i in range(1, 101):  
    print(fizzify(i))
```

Makes sense...okay, can we do that with Haskell?

The `fizzify` function isn't too bad...

Fizzify in Haskell

```
fizzify :: Int -> String
fizzify i
  | i `mod` 15 == 0 = "FizzBuzz"
  | i `mod` 3  == 0 = "Fizz"
  | i `mod` 5  == 0 = "Buzz"
  | otherwise     = show i
```

Okay, now for the main part. How do we print the `fizzify` of every integer?

Haskell FizzBuzz finished

```
import Control.Monad

fizzify :: Int -> String
fizzify i
    | i `mod` 15 == 0 = "FizzBuzz"
    | i `mod` 3  == 0 = "Fizz"
    | i `mod` 5  == 0 = "Buzz"
    | otherwise = show i

main :: IO ()
main =
    [1..100] `forM_` (\i ->
        putStrLn $ fizzify i
    )
```

That's it. Notice how similar it looks to the Python.

Explaining the main

Let's talk more about this part:

```
[1..100] `forM_` (\i ->  
    putStrLn $ fizzify i  
)
```

`forM_` is just `mapM_` with the parameters swapped. This is the same thing:

```
(\i -> putStrLn $ fizzify i) `mapM_` [1..100]
```

We're mapping a function to every int in the list. If we just used regular `map`, we'd have:

```
[putStrLn $ fizzify 1, putStrLn $ fizzify 2, ..., putStrLn $ fizzify 100]
```

Explaining FizzBuzz in Haskell (2)

The difference between `map` and `mapM` is that `mapM` will then sequence all the elements of the list with `>>`. So we get this:

```
putStrLn (fizzify 1) >> putStrLn (fizzify 2) >> putStrLn (fizzify 3) >> ...
```

In fact, I could have just written this: `mapM_ (putStrLn . fizzify) [1..100]`

If we used `mapM`, we'd have an `IO [...]` that would print everything and return the list of strings. We don't want the list of strings because `main :: IO ()` not `main :: IO [String]`, so we use `mapM_`.

However, let's look at that Haskell again and notice how *similar* it looks to Python, despite seeming really weird at first...

FizzBuzz in Haskell (3)

```
fizzify :: Int -> String
fizzify i
  | i `mod` 15 == 0 = "FizzBuzz"
  | i `mod` 3  == 0 = "Fizz"
  | i `mod` 5  == 0 = "Buzz"
  | otherwise      = show i

main :: IO ()
main =
  [1..100] `forM_` (\i ->
    putStrLn $ fizzify i
  )
```

It's really not that different. The thing inside the for loop ends up getting sequenced with itself for every element of the list. So we end up with an `IO` that does the same thing as a "for-each".

But `forM_` is a function

And this is the important thing. Haskell does not have loops as a built-in feature. They are not a part of the language. `forM_` is a function!

Its type, when applied to a list, is: `forM_ :: Monad m => [a] -> (a -> m b) -> m ()`

That is, it's a function that takes a list and a function that produces monads, and it will produce a single monad that is executed for its effects (i.e., doesn't return anything).

But notice: this is a function that works *on any monad*. We've only really been using one kind of monad: `IO`. It turns out there are many. Let's learn about some!

But first...

The quiz

Your upcoming quiz will only focus on the `IO` monad.

For the upcoming test and final, we will broaden that to other monads: `Maybe`, `Either`, `List`, `Writer`, `Reader`, or a custom monad that you or I define.

You will not be tested on the state monad or on monad transformers (covered soon).

The quiz will ask you to write a Haskell program to do some `IO`.

Let's do some practice quizzes for the `IO` monad.

Practice Quiz 1

Write a Haskell function that takes a list of names, and prints the canonical lyrics of the song "Who stole the cookies from the cookie jar?".

One verse goes like this:

```
Chorus: Who stole the cookies from the cookie jar?  
Accuser: [person 0] stole the cookies from the cookie jar!  
[person 0]: Who, me!?  
Chorus: Yes, you!  
[person 0]: Couldn't be!  
Chorus: Then who!?
```

After the verse, the next verse has `[person 1]` instead of `[person 0]`. Verse 3 will have `[person 2]`. The verses continue until the list of people is empty.

Include the types of all functions you define.

Practice Quiz 1 answers

```
cookieInvestigationNotesOneVerse :: String -> IO ()
cookieInvestigationNotesOneVerse name = do
    putStrLn "Chorus: Who stole the cookies from the cookie jar?"
    putStrLn $ "Accuser: " ++ name ++ " stole the cookies from the cookie jar!"
    putStrLn $ name ++ ": Who, me!?"
    putStrLn "Chorus: Yes, you!"
    putStrLn $ name ++ ": Couldn't be!"
    putStrLn "Chorus: Then who!?"

cookieInvestigationNotes :: [String] -> IO ()
cookieInvestigationNotes names =
    names `forM_` cookieInvestigationNotesOneVerse
```

Suggestion: rewrite this to use only one function. Instead of

`cookieInvestigationNotesOneVerse`, use a lambda function. Notice how similar it looks to an imperative language. Now write it with `mapM_`. Can you make it point free?

Practice Quiz 2

Write a Haskell program (consisting of at least `main`) in which:

- The program asks the user to input a non-negative integer
- Use `readMaybe :: Read a => String -> Maybe a` to convert the `getLine` input into an integer.
- If the given input is not a valid integer (`readMaybe` returns `Nothing` or it's not non-negative), ask again. Keep asking until it is valid.
- Print all of the squared natural numbers that are less than or equal to the number the user provides. Print each one per line.

So if the user enters `10`, the result would be `1`, `2`, `4`, `9`, all on separate lines.

Include types on all functions you define, including `main`.

Practice quiz 2 answers

```
readNat :: IO Int
readNat = do
  line <- getLine
  let result = readMaybe line :: Maybe Int
  let onError = putStrLn "invalid. must be int >= 0." >> readNat
  case result of
    Nothing -> onError
    Just x -> if x >= 0 then return x
              else onError

squares = (^2) <$> [1..]

main :: IO ()
main = do
  beneath <- readNat
  let squaresBeneath = takeWhile (<= beneath) squares
  forM_ squaresBeneath print
```

Practice quiz 2 commentary

I got a little fancy with this one. Originally I repeated `putStrLn "error. must be int >= 0."` for both the `Nothing` case and the `else` branch.

`onError` is not a function, it's an action. I create it by composing a `putStrLn` with the action I'm defining (`readNat`).

This action has type `IO Int` , which is why I'm allowed to return it. I'm returning an action that says "print an error message and then run the same action again".

Practice quiz 3

Write a Haskell program that reads an input `n` (you may assume it's a valid natural number)

Then it will draw a sideways pyramid of `#` s with the longest span having `n`.

For example, if `n = 3`, this pyramid will be drawn:

```
#  
##  
###  
##  
#
```

You must use one of `forM`, `forM_`, `mapM`, or `mapM_`. Include types of all functions.

Practice quiz 3 answer

```
main :: IO ()
main = do
  n <- (read <$> getLine) :: IO Int
  forM_ ([1..n] ++ reverse [1..(n - 1)]) (\l ->
    putStrLn $ l `stimes` "#")
  )
```

This one was shorter because I remembered that strings are monoids, so I could use `stimes` to generate the string. If you made a function that did that instead, the program would be a little longer.

Questions?

Maybe is a monad

Remember `Maybe` ? A `Maybe a` *might* be `Nothing` , but it also could be `Just` an `a` .

`Maybe` is a functor. If we `fmap` a function with a `Maybe` , it applies the function to the value inside the `Just` , and just returns `Nothing` if the value is a `Nothing` .

`Maybe` is also an `Applicative` . a `Maybe f` applied to a `Maybe x` is `Just $ f x` if both maybes are `Just` , and `Nothing` if either is `Nothing` .

And `Maybe` is a `Monad` , too.

What does that mean it can do?

Monad reminders

Monads are things that can *bind*. Bind is written `>>=`.

Bind means "based on the result of the previous program, use this function to create a new program that will be combined with the existing program."

So the question we ask about unfamiliar monads: what do they do as programs? What happens when we bind them?

Think of bind as if, in C, you could redefine what a semicolon means. You could make it, for example, check whether the previous operation was successful to change based on what it returned. This is why monads are such a powerful pattern: they are programmable semicolons. They let you create your own language within Haskell.

So what does `Maybe`'s bind do?

The Maybe Monad

Let's set the stage. Consider a bunch of computations that can fail:

```
fallableComputation1 :: Int -> Int -> Int -> Maybe Int  
fallableComputation2 :: Int -> Maybe String  
fallableComputation3 :: String -> Maybe Double
```

All of these functions return `Maybe` s, because some of their inputs might make them fail. Suppose we want to use all of them to calculate something, but we want to stop if any of them fails (like an early return).

The Maybe Monad (2)

We saw with applicative functors that we could sequence these, so that if any of them failed, we would get `Nothing`:

```
fallableResult :: Maybe Double
fallableResult =
    fallableComputation1 1 2 3 *>
    fallableComputation2 4 *>
    fallableComputation3 "hi"
```

And since `*>` is equivalent to `>>`, we can use `do` notation. Is this right?

```
fallableResult = do
    fallableComputation1 1 2 3
    fallableComputation2 4
    fallableComputation3 "hi"
```

Not quite

That isn't quite what we want! It will compile, but it probably isn't what we want.

[Why? What's wrong?]

The problem

The problem is that we had to know the arguments of the functions in advance.

There's really no point, then. We could replace the `Maybe` with an `and`.

But what we (probably) want is to have the second computation depend on the result of the first, and the result of the third computation depend on the result of the second.

Currently, we're not using that kind of dependency. In fact, `*>` *can't* do that. Only `>>=` (or the `<-` in do notation) can.

Let's see...

Example of solution

```
fallableResult :: Maybe Double
fallableResult = do
  x <- fallableComputation1 1 2 3
  y <- fallableComputation2 x
  fallableComputation3 y
```

or, equivalently:

```
fallableResult =
  fallableComputation1 1 2 3 >>=
  fallableComputation2 >>=
  fallableComputation3
```

Here, we're calling `fallableComputation1` with the initial arguments, but `fallableComputation2` and `fallableComputation3` depend on results from the previous computations.

Why is that helpful?

Because it gives us early returns.

In C, we often like this feature. It lets us return early so we can focus on the "happy path" (i.e., the code which we want to run rather than the boring error-handling code) without indenting it.

Consider this simple problem:

Parse an IPv4 address such as 123.123.123.123 into a big-endian integer (`Word32`).

In C, we might return an integer so that we could report failure, or maybe a boolean status code along with writing to a char pointer if successful...

Happy-path programming: one approach

```
// split_on_dots("123.12.1.2") == {"123", "12", "1", "2"}
// writes the number of individual strings to number_of_elements
char** split_on_dots(char* str, int* number_of_elements);
bool parse_uint32(char* str, uint32_t* i); // returns false if it fails
_Bool parse_ipv4(char* ip_str, int* result) { // *result must be 0 to begin with
    int size;
    char** splitted = split_on_dots(ip_str, &size);
    if (size != 4) return false; // the first error
    // now, if *any* of the 4 integers fail to parse, we also bail
    for (uint32_t i = 0; i < 4; i++) {
        uint32_t val;
        if(!parse_int(splitted[i], &val)) return false;
        if(val > 255) return false;
        *result |= val << ((3 - i) * 8)
    }
    return true;
} // omitted: free the strings in splitted and the array itself
```

Why happy path?

Notice that in the previous slide, we have a bunch of checks that return `false` as soon as there's an error.

This is nice, because it allows us to focus on the primary intended code path. We can always "clean up" the condition that would cause it to break, so we no longer have to worry about it anymore.

Here's what the code would look like if we *didn't* do that...

Non-happy-path-style C

```
_Bool parse_ipv4(char* ip_str, int* result) { // *result must be 0 to begin with
    int size;
    char** splitted = split_on_dots(ip_str, &size);
    if (size == 4) {
        // now, if *any* of the 4 integers fail to parse, we also bail
        for (uint32_t i = 0; i < 4; i++) {
            uint32_t val;
            if(parse_int(splitted[i], &val)) {
                if(val <= 255) {
                    *result |= val << ((3 - i) * 8)
                } else return false;
            } else return false;
        }
        return true;
    }
    else return false;
} // omitted: free the strings in splitted and the array itself
```

Non-happy-path-style C (2)

Notice that when we don't "take care" of errors earlier, we end up having giant pyramids of if-statements.

We also have to remember "okay this is the branch where we didn't have it work correctly so I need to return false".

It's just kind of annoying.

Anyway, Haskell doesn't have built-in early-returns from functions.

Why?

One of the programming-design reasons is that they would break *compositionality*, which is the feature that the meaning of an expression is only determined by the meanings of its subexpressions and the combining syntax. Early returns are like a goto that do something based on the surrounding environment rather than representing a meaningful expression by themselves.

Early returns always return from the current function, so if we wrapped them into a sub-function to refactor, the behavior of our program would change.

You might have thought it was because of lazy evaluation, but there's actually no reason we can't have lazy evaluation and early returns. Actually, many functional programming languages with eager evaluation also don't support early returns for the reason above.

So Haskell is just lame then?

No, you can actually use the `Maybe` monad to add early returns into the language, but in a "well-behaved" way. There are some neat benefits that provides, but let's see the code, first.

```
splitOnDots :: String -> [String]
parseWord32 :: String -> Maybe Word32
parseIpv4 :: String -> Maybe Word32
parseIpv4 str = do
    let splitted = splitOnDots str
    if length splitted /= 4 then Nothing else Just ()
    parsed <- mapM parseWord32 splitted -- if any are `Nothing`, early returns
    if any (> 255) parsed then Nothing else Just ()
    let shifted = zipWith shiftL parsed [24, 16, 8, 0]
    Just $ foldl1 xor shifted
```


Line by line explanation

```
let splitted = splitOnDots str
```

`splitted` is just a list of strings, each of the numbers in the ip address.

```
if length splitted /= 4 then Nothing else Just ()
```

This is the first early return. `Nothing` will cause everything after to be ignored.

We don't need to use this value, it just needs to be here so we don't return, that's why there's only `()` in the `Just`.

Line by line (2)

```
parsed <- mapM parseWord32 splitted
```

`mapM` is like `map`, but it sequences the monad afterwards. If we had just `map parseWord32 splitted`, we would have a `[Maybe Word32, Maybe Word32, Maybe Word32, Maybe Word32]`.

`mapM` then inserts a `*>` in between each of those `Maybe` s. So if any of the parts of the ip address, the whole result is `Nothing`. Otherwise, the result is `Just [Word32, Word32, Word32, Word32]`

`parsed <-` of a `Maybe [Word32, Word32, Word32, Word32]` will either skip the entire remainder of the `do` block and become `Nothing` when the `Maybe` is `Nothing`, or `parsed` will become the list if it exists. This means that we early return automatically without an `if`.

Line by line (3)

```
if any (> 255) parsed then Nothing else Just ()
```

This is another early return (the third if we count `<-`), at this point, we can be sure that all of the values inside the list are valid bytes.

```
let shifted = zipWith shiftL parsed [24, 16, 8, 0]
```

Here we just shift the first byte left by 24, the second by 16, etc.

Just `$ foldl1 xor shifted` the `foldl1 xor` xors the words together to make one big word. `foldl1` is like `foldl`, but it uses the first element of the list

Cleaning it up

Do we really have to write `if condition then Nothing else Just ()` every time?

No, we can use a function: `when :: Applicative f => Bool -> f () -> f ()`

This function is very simple:

```
when condition val = if condition then val else pure ()
```

Remember that `pure` is just a constructor for `Applicatives`. For the `Maybe` applicative, `pure = Just`. So `pure () == Just ()`.

Now we can do this:

```
when (length /= 4) Nothing
```

Practice: can you define a function `abortIf` which takes a predicate and early returns from a `Maybe` if the predicate is true? Give its type and definition.

Questions?

How does it work?

That's how we *use* the `Maybe` monad to add early returns to the language.

But how do we *create* the `Maybe` monad?

That is:

```
instance Monad Maybe where  
    ...
```

What goes in the `instance` definition?

What are the functions that have to be there for monads?

How it works

First, the two functions are `return` and `>>=` (bind)

```
instance Monad Maybe where
  return = pure
  m >>= f = ...
```

`return` is just a constructor for monads. It has *nothing* to do with early returns. In: `if condition then Nothing else return ()`, `return ()` is just a synonym for `pure ()`, which is a synonym for `Just ()`.

Notice that `return ()` means "keep going"! It's the `Nothing` that means to return early. This is confusing and unfortunate, but the name `return` was chosen for this function because often it's the last thing you do in a `do` block. In our example, we could have used `return` instead of `Just`: `return $ foldl1 xor shifted`

What about `>>=` (bind)?

This is the key. Remember the type of this function:

```
(>>=) :: Monad m => m a -> (a -> m b) -> m b
```

So, for the `Maybe` monad: `(>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b`

This type means "take a maybe and a function. The function will receive the value inside the maybe if it exists and then return a new maybe"

The bind operator for `Maybe` is supposed to check if the maybe being bound is `Nothing`, and if so, return `Nothing`. Otherwise call the given function and pass the value inside the given `Just` to it.

[Can you write this?]

One version

I named it `Maybe'` so that Haskell would not complain about name conflicts.

```
instance Monad Maybe' where
  return = pure -- remind me what this is for?
  Nothing' >>= _ = Nothing'
  Just' x >>= f = f x
```

(remember, if you do this yourself, `Maybe'` must also be an `Applicative`, which means it must also be a `Functor`. Try to recall how to make it one of both.)

Let's take a bit to understand this:

- If the thing to the left of the `>>=` is `Nothing'`, return `Nothing'`
- If the thing to the left of the `>>=` is `Just' x`, call the given function on the `x`.

Questions?

What if we want something even in failure?

Sometimes, when some code fails, we don't just want `Nothing`, we want a proper error.

That's what the `Either` monad is for.

`Either` is a data type that stores one of two values:

```
data Either a b = Left a | Right b
```

So an `Either Int String` could either be a `Left 20` or a `Right "hello"` among many other possibilities.

So, how can `Either` be a `Functor` or `Applicative`?

Either as a Functor

What happens when we use `Either` as a functor? ([Source](#))

```
instance Functor (Either a) where
  fmap _ (Left x) = Left x
  fmap f (Right y) = Right (f y)
```

First, notice that we didn't write `instance Functor Either`, we wrote `instance Functor (Either a)`.

Remember, a `Functor` must have kind `* -> *`. But `Either` has kind `* -> * -> *`. It takes two type arguments, i.e., `Either Int String`, not just `Either Int`.

This brings us to the second point: the `a` type is the one we don't really care about.

`Right` is the "good" constructor, for non-error values (because it's "right", i.e., correct).

`Left` is the constructor for errors. So if it's left, just return it, don't apply the function.

Either as an Applicative

```
instance Applicative (Either e) where
  pure      = Right
  Left  e <*> _ = Left  e
  Right f <*> r = fmap f r
```

([Source](#))

It works basically the same as `Maybe`, with `Left` taking the place of `Nothing` and `Right` taking the place of `Just`.

If it's `Left`, we ignore the right hand side.

If it's `Right f <*> Left x`, we end up returning `Left x` because of `fmap`.

Only if it's `Right f <*> Right x` do we return `Right $ f x`

Either as a Monad

As a monad, `Either`'s bind is pretty much the same as `Maybe`.

If it's `Left`, we return that. We ignore everything after it.

If it's `Right`, we feed the value to the function and continue.

Can you write it?

```
instance Monad (Either a) where
    return = pure
    ???
```

Either's bind

```
instance Monad (Either a) where
  return = pure
  Left x >>= _ = Left x
  Right x >>= f = f x
```

If there are a chain of `Either` s, it returns the first one that is `Left` . This gives you a way to early return with an error:

```
Right 10 >> Left "Oops" >> Right 20 == Left "Oops"
```

```
do -- if either function call returns Left, return that error.
  result <- someFallablefunction
  result2 <- someOtherFallablefunction result
  return $ result2
```

Early return with `Either`

With maybe, we could early return, but not provide an actual value.

With either, we can early return anything. It's mainly intended for errors, since once an error happens you don't want to continue.

Let's make a password checker. If a password is bad, it returns why.

```
goodPassword :: String -> Either String ()
goodPassword pwd = do
  when (atLeastOneUpper pwd) (Left "Must have at least one uppercase letter")
  when (atLeastOneLower pwd) (Left "Must have at least one lowercase letter")
  when (atLeastOneSymbol pwd) ...
```

If we get through all the checks, we just end up with `Right ()`, indicating the password is good. But if we fail, we actually know why.

Knowledge check for `Maybe` and `Either` a

1. Write a function named `divideAll :: [Double] -> Maybe Double` that takes a list of `Double` s and divides them over and over, so that `[1.0, 2.0, 3.0, 4.0] == Just $ 1.0 / 2.0 / 3.0 / 4.0` , however, if any but the first number is `0` , the result is `Nothing` . If the list is empty return `1.0` .
2. Write that same function but have it return an `Either String Double` and have it return the message `"divide by zero"` in the event that there is a zero anywhere but in the first index.
3. Write a Haskell program that reads a password and then applies the following checks: `atLeast12chars` and `atLeastOneDigit` . If a check fails, return a `Left` with an error message. If it succeeds, return the password.
4. Refactor that program so that each check does only one thing and returns `Either` .

Maybe/Either KC answers (1)

```
divideAll :: [Double] -> Maybe Double
divideAll [] = Just 1
divideAll (x : xs) = do
    when (0 `elem` xs) Nothing
    Just $ foldl' (/) x xs
```

```
divideAll' :: [Double] -> Either String Double
divideAll' [] = Right 1
divideAll' (x : xs) = do
    when (0 `elem` xs) $ Left "divide by zero"
    Right $ foldl' (/) x xs
```

```
checkPassKc :: String -> Either String String
checkPassKc str = do
    when (length str < 12) $ Left "must be at least 12 characters"
    unless (any isDigit str) $ Left "must contain a digit"
    Right str
```

Maybe/Either KC answers (2)

```
checkLengthReq :: String -> Either String String
checkLengthReq str =
    if length str < 12
    then Left "must be at least 12 characters"
    else Right str
```

```
checkDigitReq :: String -> Either String String
checkDigitReq str =
    if not (any isDigit str)
    then Left "must contain a digit"
    else Right str
```

```
checkPassKc' :: String -> Either String String
checkPassKc' str = do
    checkLengthReq str
    checkDigitReq str
    Right str
```

Questions?

Why are we doing this?

So far we've seen several uses for monads:

- The `IO` monad lets us describe computations that call external code and pipe the results to other computations that call external code.
- The `Maybe` monad lets us early return if there's no useful result.
- The `Either` monad lets us return errors.

Why are we doing this? (2)

Fundamentally, monads are about letting you add your own language as to how actions or statements are composed.

Imagine if, in C, you could say "hey, for every statement in this function, if it fails, I want you to make the whole thing fail"

A monad is like a custom programming language. You describe how statements in the language are combined.

In fact, sometimes people will use monads to define a "DSL" (domain specific language)

So far, the monads we've seen have been pretty tame. Now we're going to see some that *really* add features to the language. In particular, non-determinism, logging, dependency injection, stateful programming, as well as how to combine monads.

The list monad

It might surprise you to learn that `[]` is a monad. That's right, good old lists. But how?

The instance looks like this:

```
instance Monad ([]) where
  return = pure -- same as []
  l >>= f = concat $ map f l -- or concatMap f l
                                -- or [y | x <- l, y <- f x] (official)
```

What does that do?

It generates every combination

If we do `[1, 2, 3] >>= (\x -> [x, x, x])` we get: `[1, 1, 1, 2, 2, 2, 3, 3, 3]`

This is what happens:

1. We run the function on every element of the original list:

```
[[1, 1, 1], [2, 2, 2], [3, 3, 3]]
```

2. Then we flatten the lists of lists into a single list with `concat` : `[1,1,1,2,2,2,3,3,3]`

But why would it work that way?

It models non-determinism

Let's consider a problem. This is a Knight on a chessboard:

```
. . . . . . .  
. . * . * . .  
. * . . . * .  
. . . N . . .  
. * . . . * .  
. . * . * . .  
. . . . . . .  
. . . . . . .
```

The "N" is the knight (K means king, so we use N to disambiguate).

The start (*) represent all the places the knight is legally allowed to move in one turn.

(The unicode characters for the chess piece and the board tiles weren't rendering correctly, so please forgive the ASCII art.)

The goal

The objective is to determine all the places the knight could be after `n` turns.

Here are some answers. You already saw one turn. What about two?

```
* _ * _ * _ * _  
_ _ _ * _ _ _ *  
* _ * _ * _ * _  
_ * _ N _ * _ *  
* _ * _ * _ * _  
_ _ _ * _ _ _ *  
* _ * _ * _ * _  
_ * _ * _ * _ _
```

Take a moment to convince yourself that those stars really do represent places the knight can move after exactly two moves. (we can whiteboard it or open Lichess).

How can we do this using the list monad?

Some preliminaries

First, let's write some functions to generate a list of valid moves:

```
inBounds :: Int -> Int -> Bool
inBounds file rank =
    rank >= 1 && rank <= 8 &&
    file >= 1 && file <= 8
validMoves :: Int -> Int -> [(Int, Int)]
validMoves file rank =
    filter (uncurry inBounds) [
        (file - 1, rank + 2),
        (file + 1, rank + 2),
        (file + 2, rank + 1),
        (file + 2, rank - 1),
        (file + 1, rank - 2),
        (file - 1, rank - 2),
        (file - 2, rank - 1),
        (file - 2, rank + 1)
    ]
```

The key function

This is the main function that determines where a knight can go. This is the first time where we use the list as a monad:

```
knightCanGo :: Int -> Int -> Int -> [(Int, Int)]
knightCanGo file rank 0 = [(file, rank)]
knightCanGo file rank turns = do
    (file', rank') <- validMoves file rank -- for each valid move ...
    knightCanGo file' rank' (turns - 1) -- concatenate all the places it can go
```

We use the monad in the second definition, with `do` notation.

We generate a list with `validMoves`. However, `(file', rank')` is not a list. It's an individual `(file, rank)`. The code after that runs for every *element* of the list.
[let's dwell on how this works]

What?

In the list monad, every line of the `do` multiplies the list (a cartesian product).

```
do { [0,0,0] ; [1,1,1] } == [1,1,1,1,1,1,1,1,1]
```

For every zero, we're producing `[1,1,1]`, and they get concatenated.

```
do { [0,0,0]; [1,1,1]; [2,2,2] } ==  
[2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2,2]
```

Now there are 27 `2` s. For each `0`, we are pairing up three `1` s. By itself, that would produce `[1,1,1,1,1,1,1,1,1]`. We are then pairing each `1` up with `[2,2,2]`. So there are $9 * 3 = 27$ `2` s.

But that's just a fancy way of multiplying the list. we could easily just do `9 `stimes` [2,2,2]`. That would also give 27 `2` s. So why?

Why?

The main value of `List` being a `Monad` is that we can actually use the values inside of it to produce new values.

When we use the bind operation, the rest of the monad runs for every element:

```
do { x <- [1,2,3] ; [x,x,x] } :: [Int] == [1,1,1,2,2,2,3,3,3]
```

After a `x <-`, we run the whole `do` block for each option for `x`. This ends up doing:

1. First, let `x` be `1`, generate `[1,1,1]`
2. Then, let `x` be `2`, generate `[2,2,2]`
3. Then, let `x` be `3`, generate `[3,3,3]`
4. Finally, take `[[1,1,1], [2,2,2], [3,3,3]]` and concatenate.

Filtering with the list monad

What if we had an if-expression to determine if we wanted to combine a list or not?

```
do { x <- [1..10] ; if even x then [x] else [] } == [2,4,6,8,10]
```

This will give us only the even numbers.

Compare with this:

```
[ x | x <- [1..10], even x ] == [2,4,6,8,10]
```

Notice how similar list comprehensions are...

Compare with list comprehensions

Fun fact: in early Haskell versions (between 1997 and 1999) you could use list comprehensions with any Monad*. It sounds like it was just a variant of `do` notation.

I left a citation because the history of Haskell is rather interesting.

* Paul Hudak, John Hughes, Simon Peyton Jones, and Philip Wadler. 2007. A history of Haskell: being lazy with class. In Proceedings of the third ACM SIGPLAN conference on History of programming languages (HOPL III). Association for Computing Machinery, New York, NY, USA, 12–1–12–55.

<https://doi.org/10.1145/1238844.1238856>

Random note about <-

Anytime we use the <-, from a type perspective, we're "peeling off" one layer of monad.

So:

```
someList :: [(Int, Int, Int)]
someList = do
  x <- someOtherList -- some other list is [Int]
                      -- but x is an Int.
  return (x, x, x)
```

For another monad: `getLine :: IO String`, in `do { x <- getLine; ... }`, `x` is a `String`. `<-` is the closest thing we have to "getting the value out" of a monad. Alternatively `>>=`.

Questions?

Writing things out

In class you may have seen me use `trace` or `traceShow` to print debugging output.

This is a kind of non-functional escape hatch that lets us do print debugging.

Sometimes, it's just really useful to be able to do that but we actually want to use the thing that we wrote. But `trace` won't let us access the string: that would violate functional purity.

But sometimes it really is nice to be able to be able to track a list of things in a "stateful" way (i.e., pushing onto the list). And because of monads, we don't have to give that up.

Specifically, the `Writer` monad lets us do this.

The writer monad ([link](#))

```
newtype Writer w a = Writer { runWriter :: (a, w) }
    -- the 'w' is the type of the "log", the 'a' is the return type
instance Monoid w => Monad (Writer w) where -- the log must be a monoid
    return = pure -- we'll show the applicative instance in a second

(Writer (a, w)) >>= f =
    let Writer (b, w') = f a
    in Writer (b, w <> w')
```

(note: We can construct a Writer directly with `Writer (20, "hello")`, we don't have to write `Writer { runWriter = (20, "hello" }`. Record syntax allows this as a shortcut.)

(note2: this isn't the real source code. `Writer` is actually defined in terms of `WriterT`, which is a monad transformer. We'll talk about those in a bit. This is how it would work without monad transformers, though.)

The writer monad (2)

Fundamentally, `Writer` is a return value (type `a`) together with a monoid (`w`).

The monoid is used to accumulate "logged" values. The `a` is just...whatever you want it to be. The `a` does not have to interact with `w` at all.

So if a function returns a `Writer String Int`, that means it really returns a `(Int, String)` (the order is swapped in the record).

So what's the point? Why would we write a function to have this type:

```
someFunction :: Int -> Int -> Writer String Int
```

instead of this:

```
someFunction :: Int -> Int -> (Int, String) or even Int -> Int -> Int
```

?

Because of Bind

Because when we bind a writer with the result of a function, we combine the monoids:

```
(Writer (a, w)) >>= f =  
  let Writer (b, w') = f a  
  in Writer (b, w <> w')  -- remember, <> combines monoids
```

The basic idea of `Writer` is: "this is an ordinary value but it also has a log or accumulator or something with it"

This accumulator is "write only". It's a monoid so we can just keep `<>` data to it, to grow the accumulator.

We `import Control.Monad.Writer.Lazy` to get access to the `Writer` monad.

tell

```
tell :: Writer w m => w -> m ()
```

It's a function that takes a monoid `w`, and produces a `Writer` with that monoid in it.

If you sequence two `Writer` s, from the `>>=` implementation, you can see that their monoids get combined with `<>`.

So this prints `6`:

```
import Control.Monad.Writer.Lazy
someWriter :: Writer (Sum Int) ()
someWriter = do
    tell (Sum 1)
    tell (Sum 2)
    tell (Sum 3)
```

```
-- the snd is because runWriter returns the result () and the sum
main = print $ getSum $ snd $ runWriter someWriter
```

Writer for logging

What about that initial idea? Logging? It's easy, because `String` is already a `Monoid`.

```
someCalculationWithLogging :: Int -> Writer String Int
someCalculationWithLogging start = do
    let x = start * 2
    tell $ "doubling start to get " ++ show x ++ "...\\n"
    let y = x + 1
    tell $ "adding 1 to get " ++ show y ++ "...\\n"
    let z = y * 3
    tell $ "tripling to get " ++ show z ++ "...\\n"
    tell $ "final result: " ++ show z ++ "\\n"
    return z
main = do putStrLn $ snd $ runWriter $ someCalculationWithLogging 2
```

Caution: remember that evaluation is lazy. Just because we compute `x` and then log does not mean the log happened after. In this case, we forced it with `x` in the log.

Imperative sum with `Writer`

Remember how we learned about `foldl` ? And how we could write `sum = foldl (+) 0` ? That's nice, but if you really wanted to do it the imperative way, you could.

```
imperativeSum :: [Int] -> Int
imperativeSum list =
  let (_, Sum result) = runWriter $
    forM_ list (\item ->
      tell (Sum item)
    )
  in result
```

`runWriter` gets the monoid and result (which is `()`) out of the `Writer`. But what about `tell` ? `tell` is the function that let's us "write to" the `Writer`. If we say `tell (Sum 2)`, we're adding `2` to the existing `Sum` monoid.

MonadWriter

If you actually type `:t tell` into `ghci`, you actually get this type:

```
tell :: MonadWriter w m => w -> m ()
```

`MonadWriter` is a typeclass that is a `Monad`. `Writer` is an instance of `MonadWriter`.

Why? Because there is something called `WriterT`, which is something called a "monad transformer". Both `Writer` and `WriterT` are instances of `MonadWriter`.

We'll talk more about them towards the end of this module.

For now, just note that if you see a typeclass `MonadWriter`, `Writer` will work. Likewise, `MonadIO` has `IO` as an instance.

Remember that monads are wrappers

A `Writer String Int` is a writer that will do some computations, which will log or write to a `String`, and then return an `Int`.

The first value is the monoid that we're `tell` ing to.

The second value is the result of the computation.

This is true of every monad: the last value in its type is the "return" value. It's the result. If you write `return 7`, you will create a monad that has that value in its "return slot".

So think of

Knowledge Check

1. Write a function with the type `String -> Writer [Int] ()`, where the monoid stores the index of each `'a'` in the string.
2. Rewrite that function to not use `Writer`, and to have a type `String -> [Int]`.
3. Now, go back to using `writer`, but use `runWriter` to get the monoid out of the writer and return it, so that the signature is `String -> [Int]` like in problem 2.
4. Compare and contrast. Which did you find easiest? Did you notice any similarities?

Writer knowledge check answers

```
aIndices :: String -> Writer [Int] ()
aIndices str = do
    forM_ (zip [0..] str) (\(i, c) ->
        when (c == 'a') (tell [i])
    )
```

```
aIndices' :: String -> [Int]
aIndices' str = fst <$> filter ((== 'a') . snd) (zip [0..] str )
-- or
aIndices' str = fst <$> filter (\(i, c) -> c == 'a') (zip [0..] str)
```

```
aIndices'' :: String -> [Int]
aIndices'' str = execWriter $ -- execWriter means (snd . runWriter)
    forM_ (zip [0..] str)
        (\(i, c) -> when (c == 'a') (tell [i]))
```

Writer knowledge check compare and contrast

First, make up your own mind!

Don't read on until you have thought about it and formed an opinion.

It's important to practice doing this. Make a hypothesis, then test it. Make an opinion, then evaluate it. Don't read passively and wait for me to tell you what to think. Your opinion is valid too, and it might be different than mine.

Writer knowledge check compare and contrast (2)

My opinion is that both are readable, but I prefer the non-writer way as long as we use a lambda expression instead of the composition with `snd`.

I like it because we can read from right to left. "Start with a `str`, then `zip` it with the natural numbers, then keep only the pairs with an `'a'`, then select only the first element of the pairs (the index). It's clear that it returns the indices.

Compare and contrast (3)

I don't hate the writer though, it is basically imperative. Compare:

```
def aIndices(str):  
    result = []  
    for (i, c) in enumerate(str):  
        if c == 'a': result.append(i)  
    return result
```

```
aIndices'' :: String -> [Int]  
aIndices'' str = execWriter $  
    forM_ (zip [0..] str)  
        (\(i, c) -> when (c == 'a') (tell [i]))
```


Writer summary

Monads give you the ability to kind of extend the language. In this case, `Writer` gives us *write-only* accumulator "result" values we can append or add to.

A `Writer` is basically an `(a, w)`, where the `w` is a monoid, and the `a` is whatever you want it to be.

A function that returns `-> Writer String Int` is very similar to a function that returns `-> (Int, String)`.

However, if we put a `Writer String a` besides a `Writer String b` in a `do` block, the compiler will automatically `++` the two strings together.

Writer summary (2)

If we want to manually concat some data to the writer, we can use `tell`.

`tell` will take a monoid and `<>` it with whatever is in the current writer we are building (often in a `do` block).

If all of the values you want to `<>` follow a pattern, you probably don't need `Writer`. But if they are irregular (like logging messages), `Writer` and `tell` can be very useful.

It's also useful when doing the equivalent of a `for` loop with an accumulator.

To get the monoid out of the writer, we can use `execWriter`.

Remember, a `Writer` is a wrapper around an `(a, w)`. `runWriter` pulls that tuple out. `snd . runWriter` pulls the `w` out of the tuple. `execWriter = snd . runWriter`

Questions?

Readers

So there's a `Writer` monad. Is there a `Reader`?

Yup. Instead of a `tell` function we use to write to a write-only value, we use `ask` to retrieve a read-only value.

Here's a simple example:

```
import Control.Monad.Reader -- no need for .Lazy. Reader is always lazy.
greetUser :: Reader String String
greetUser = do
  name <- ask
  return $ "hello, " ++ name ++ "!\n"
```

`greetUser` represents a computation that will produce a `String`, but which depends on a `String`. It's very similar to a `String -> String`, but the first argument is produced with `ask`.

Why Reader?

Let's see some more code to maybe understand why a bit better:

```
waitingForUser :: Reader String String
waitingForUser = do
  name <- ask
  return $ "waiting for " ++ name ++ " to respond...\n"
logUser :: Reader String String
logUser = do
  name <- ask
  return $ "user " ++ name ++ " is registered.\n"
```

These are two more `Reader` s. They print different messages based on whatever name they are given. `ask` is a monad that produces the name, given in the future (we don't know it yet).

Why Reader (2)

Here's a function where we use all the readers:

```
loginMessage :: Reader String String
loginMessage = do
  greeting <- greetUser  -- "run" the reader monad and get the result.
  log <- logUser  -- these monads *automatically* have the string passed to them
  waiting <- waitingForUser
  return $ greeting ++ log ++ waiting ++ "$>"
```

Notice, this still returns a `Reader`. At no point do we actually know the user's name. But we are able to cleanly produce a prompt that will work when we provide the name at some point in the future.

Finishing it up

Here's the code that actually finally prints the prompt. Here, we use `runReader`, which lets us provide a name:

```
main = putStrLn $ runReader loginMessage "alice"
```

This prints:

```
hello, alice!  
user alice is registered.  
waiting for alice to respond...  
$>
```

Really, why though?

The reader monad is a little weird in that most of its useful value comes from composition, rather than `do` notation. This does the same thing as `loginMessage`:

```
loginMessage' :: Reader String String
loginMessage' =
    concat <$> sequence [ greetUser, logUser, waitingForUser, pure "$>" ]
```

Basically, it's useful when you have a bunch of functions that all take the same argument, such as:

1. A database connection that is used in many places in a backend web app.
2. Some configuration data that an app needs in many places.
3. A group of global constants

Reader source

```
data Reader r a = Reader { runReader :: (r -> a) }
instance Functor (Reader r) where
    fmap f (Reader g) = Reader (f . g) -- just apply f to the result
instance Applicative (Reader r) where
    pure a = Reader (const a) -- a reader that ignores its read value
    Reader f <*> Reader x = Reader (\r -> f r $ x r)
instance Monad (Reader r) where
    return = pure
    m >>= f = Reader $ \r -> runReader (f (runReader m r)) r
    -- ^^ here we create a new reader that first runs the old one and then
    -- runs the function on the result, creating a new reader
```

It's just a wrapper around a function. It lets us chain a bunch of a compositions that all need to use the same value (called `r`).

(Note, again, in actuality, `Reader` is defined in terms of `ReaderT`, but its definition is equivalent to what I have above.)

Reader practice question

Suppose we have these data types

```
data Db = HugeConnectionObject { ... }
```

1. Suppose we have 2 readers, `selectStudents :: Reader Db [String]` which gets the names of students, and `selectEmployees :: Reader Db [String]` which gets the names of employees.

Write a `Reader`, `selectAllPeople :: Reader Db [String]`, which will combine both of the previous readers above. You may assume both lists are completely distinct.

2. Suppose you actually have a `Db` named `db`. How can you provide this value to the reader to actually get the `[String]` out of it?

Reader practice answer

1.

```
selectAllPeople :: Reader Db [String]
selectAllPeople = do
  students <- selectStudents
  employees <- selectEmployees
  return $ students ++ employees
```

or

```
selectAllPeople = concat <$> sequenceA [selectStudents, selectEmployees]
```

2.

```
main = do { db <- connectToDb ; let ppl = runReader selectAllPeople db ; ... }
```

Questions?

Exam practice problems

Remember that on the term exam, I can test you on `IO` , `Maybe` , `Either` , and `[]`

On the final, I can test you on any of those, plus `Reader` , `Writer` , and potentially making your own monad.

I won't test you on `State` or on monad transformers (which are later in this lecture)

Here are some practice questions. All of them are the type of question that could be on the final, but only the first 2 could be on the 2nd term exam.

Exam practice 1

Refactor `generateReport` so that it uses `Either Error` as a monad rather than `case`.

```
type Error = String
storedCredentials :: Either Error Certificate
connectToDb :: Certificate -> Either Error Connection
selectData :: Connection -> Either Error [String]
report :: Either Error [String]
report =
  case storedCredentials of
    Left error -> Left error
    Right cert ->
      case connectToDb cert of
        Left error -> Left error
        Right connection ->
          selectData connection
```

Exam practice 1 answer

note: IRL this would use `IO` and `ExceptT`. I've simplified it to be an exam question.

```
report :: Either Error [String]
report = do
    cert <- storedCredentials
    connection <- connectToDb cert
    selectData connection
```

```
cleanerReport :: Either Error [String]
cleanerReport =
    storedCredentials >>=
    connectToDb >>=
    selectData
```

```
codeWarsTopComment = foldl (>>=) storedCredentials [connectToDb, selectData]
```

Exam practice 2

Consider a game in which every player has a PlayerName, a team, and a color:

```
type PlayerName = String
data Team = Redfor | Blufor
data Color = Green | Gold | Orange
type Assignment = (PlayerName, Color, Team)
```

`("Alice", Green, Redfor)` means that player "Alice" has been assigned as the Green player for the Redfor team.

Define a function `allAssignments :: [PlayerName] -> [Assignment]` such that it uses the list monad to generate every possible assignment between a list of playernames and the colors and teams they could have. Sample input output on next page. Then, in `main` print every assignment for `["Ally", "Bob"]` on its own line.

You **must** use the list monad to receive credit. The order of results doesn't matter.

Exam practice 2 sample input/output

```
allAssignments ["Alice", "Bob"] ==  
[  
  ("Alice",Green,Redfor),  
  ("Alice",Gold,Redfor),  
  ("Alice",Orange,Redfor),  
  ("Alice",Green,Blufor),  
  ("Alice",Gold,Blufor),  
  ("Alice",Orange,Blufor),  
  ("Bob",Green,Redfor),  
  ("Bob",Gold,Redfor),  
  ("Bob",Orange,Redfor),  
  ("Bob",Green,Blufor),  
  ("Bob",Gold,Blufor),  
  ("Bob",Orange,Blufor)  
]
```

Exam practice 2 (answer)

```
allAssignments :: [PlayerName] -> [Assignment]
allAssignments players = do
  player <- players
  team <- [Redfor, Blufor]
  color <- [Green, Gold, Orange]
  return (player, color, team)

main :: IO ()
main = mapM_ print $ allAssignments ["Alice", "Bob"]
```

Exam practice 3

Write a program with `Writer` that imperatively calculates a factorial with `forM_/mapM_`. That is, define this function: `impFact :: Integer -> Writer (Product Integer) ()`

Then define `main` so that it reads in a natural number (which you may assume to be well-formed) and computes the factorial of it using your function.

You must use the `Writer` monad for `impFact` (even if it's not more elegant than just using regular functions)

Hints:

1. You can use `execWriter :: Writer w a -> w` to get the monoid out of the writer.
2. You can use `getProduct :: Product a -> a` to get the product out of the monoid.

Exam practice 3 answer

```
impFact :: Integer -> Writer (Product Integer) ()
impFact n =
    forM_ [1..n] $ \i ->
        tell $ Product i

-- alternatively (cleaner):
impFact' :: Integer -> Writer (Product Integer) ()
impFact' n = mapM_ tell (Product <$> [1..n])

main :: IO ()
main = do
    n :: Integer <- read <$> getLine
    print $ getProduct $ execWriter $ impFact' n
```

Exam practice 4

Consider this data type:

```
data Counter a = Counter a Int -- the `Int` stores a count
    deriving Functor -- fmap f (Counter x i) = Counter (f x) i

instance Applicative Counter where
    pure x = Counter x 0
    (Counter f i) <*> (Counter x j) = Counter (f x) (i + j + 1)
```

This type should be a monad that counts the number of times it binds or `<*>`s. Show the monad instance yourself (80%), specifically the `>>=` implementation.

Monads must obey identity laws: `return x >>= f == f x`, `x >>= return == x` and also the associativity law: `m >>= g >>= h == m >>= (\x -> g x >>= h)`.

Does our monad follow these laws? If not, state which one is broken.

Exam practice 4 answer

The answer depends on your bind implementation. If you implemented it like this:

```
instance Monad Counter where
  (Counter x i) >>= f =
    let (Counter y j) = f x
    in Counter y (i + j + 1)
```

Then it violates the identity laws, because `Counter 7 5 >>= return /= Counter 7 6`, (it also violates the other one: `return 7 == Counter 7 0 >>= f == a counter with 1 greater count than f 7`).

If you forgot the `+ 1` in the bind, it no longer counts the number of binds, but also no longer breaks any laws. So you would lose points on the bind but be expected to say "it breaks none of the laws". Either way, associativity is preserved, because addition is associative.

Exam practice 5

You took the brave step of including Haskell in your resume in the "proficient" section. Your technical interviewer is impressed when you pass FizzBuzz in it. Then they ask you:

What if we want to extend FizzBuzz so that we can add new divisors and strings. Like "Baz" if a number is divisible by 7? I want to define a list of divisors and messages, and only print the number if it fails all of them, but to combine those that don't fail. So if we had (3, "Fizz"), (5, "Buzz"), (7, "Baz"), then 21 would print "FizzBaz", but 22 would still print "22".

Solve the problem using the writer monad within a function `Int -> String`. This function should use writer to check for factors of an `Int`, and use `tell` with a message for each one it finds. Then, it should extract the monoid to see if anything was written, replacing it with the number itself as a string if not.

Exam practice 5 answer

```
overEngineeredFizzbuzzMessages :: [(Int, String)]
overEngineeredFizzbuzzMessages = [
    (3, "Fizz"),
    (5, "Buzz"),
    (7, "Baz"),
    (11, "Quux") -- etc.
]

overEngineeredFizzbuzz :: Int -> String
overEngineeredFizzbuzz n =
    if null message then show n else message
    where
        message = execWriter $
            forM_ overEngineeredFizzbuzzMessages $ \(m, msg) ->
                when (n `mod` m == 0) (tell msg)
```


More practice?

You know what to do! Feed this presentation to an LLM and ask it to feed you more short questions to grind on. Make sure to use the raw `.md` file for the presentation because LLMs are good with markdown.

Monad knowledge only comes from using them. Things will start clicking into place.

You really do need to learn each monad. Treat them all as tiny programming languages, like the ones we learned to parse.

Once you learn all of them, you can take your knowledge to the next level. Next module we'll be talking about the `State` monad, as well as combining monad features with monad transformers!

Questions?